AGENTIC WORK BEST PRACTICES

Agentic Work Best Practices — Research Report v1

Deep-research pass v1 — 8 topics, 24 agents, adversarially verified

2026-07-12

Source of truth · SebsKk/AgenticWorkBestPractices

Executive Summary

Executive Summary

This report distills the state of the art (late 2025–2026) for running autonomous coding agents — **Claude Code** and **OpenAI Codex CLI** — safely and productively in production. The through-line is a single shift in mindset: reliability now comes from the *system you build around the model* (the loop, the checks, the context flow, the safety boundaries), not from a cleverer prompt.

What we researched, and how

We covered eight problem areas — agentic loops, lifecycle hooks, session/context management, repo setup, verification & self-checking, subagent orchestration, workstream logging, and permissions & security — plus three platform deep-dives (Claude Code configuration internals, Codex CLI internals, and the cross-cutting "harness engineering" literature).

Each topic went through two passes: a **research pass** that produced concrete, cited practices (with exact file paths, flags, and field names), followed by an **adversarial verification pass** that re-checked every claim against primary sources and issued a verdict — *confirmed* or *needs-correction* with corrected guidance. Sources were weighted toward official documentation (Anthropic Claude Code docs, the Anthropic engineering blog, the Claude Developer Platform, and OpenAI's Codex docs), supplemented by peer research (EvilGenie, Cursor's reward-hacking study, RuVerBench) and practitioner reports where official material was thin. Overall confidence is **high** across all eight topics. Notably, verification caught several "common knowledge" facts that are already wrong — strong evidence that this surface moves weekly and must be re-checked against your installed CLI version.

Headline findings by topic

- **Agentic loops.** Every unattended loop is three deliberate choices: what *starts* the next iteration (a condition, an interval, or an event), what *stops* it (a verifiable check *plus* a hard turn/dollar ceiling), and what's *allowed* unattended (sandbox + approval + branch scoping + human review). The dominant 2026 failure mode is runaway cost, so ceilings, a kill switch, token/cost observability, and a human checkpoint before anything irreversible are non-negotiable. Claude Code offers rich primitives (`/goal`, `/loop`, the Monitor tool, cloud Routines, SDK `maxTurns` / `maxBudgetUsd`); Codex covers the same ground but **has no per-run budget cap**, so Codex loops need external guards.
- **Hooks.** Hooks are the deterministic control layer — shell commands fired at fixed lifecycle points so policy is enforced by the harness rather than the model's discretion. Both CLIs have converged on a near-identical model (event-keyed config, a matcher, exit-code-2 to block). The load-bearing patterns: auto-format/lint on `PostToolUse`, block dangerous commands and protected files on `PreToolUse` (this works *even under* bypass-permissions), context injection at session start, and

`Stop`-hook completion gates that force tests to pass before the agent can quit. Because hooks run arbitrary code with your full privileges, silently, they are also a security surface.

- **Session management.** The context window is the binding constraint and quality degrades as it fills ("context rot"), so good session hygiene is active curation of the smallest high-signal token set: keep memory files lean, `/clear` between unrelated tasks (reserve `/compact` for continuity *within* one task), push heavy reads into subagents, externalize state to files that survive compaction, and resume named sessions instead of re-explaining. For programmatic agents, server-side context editing + compaction + the memory tool let long runs continue past normal exhaustion (Anthropic reports ~84% fewer tokens and up to +39% task performance).
- **Repo setup.** Making a repo "agent-ready" is a small set of committed config surfaces. Start with a lean root instructions file (under ~200 lines) that leads with exact commands and hard boundaries — and note that **Claude Code reads `CLAUDE.md`, not `AGENTS.md`**, so unify multi-tool repos via an `@AGENTS.md` import or a symlink. Add progressive disclosure (path-scoped rules, skills, nested files), a committed deny-first permission allowlist, secret/egress hardening plus OS sandboxing, MCP-as-config, and a self-runnable verification loop. Remember the split: instructions are *advisory*; permissions, hooks, and the sandbox are the *enforcement*.
- **Verification & self-checking.** An agent is only as reliable as the check it can run on itself — give it a machine-readable pass/fail and it closes its own loop; withhold one and *you* become the verification loop. Best practice layers four things: a runnable check, **runtime observation at the done-gate** (drive the real surface users touch, not just green tests), an independent fresh-context grader, and an explicit "Done when" contract. The fast-rising concern is **reward hacking**: more capable models increasingly game verifiers (hardcoding outputs, weakening assertions, editing tests, looking up upstream fixes), so guarding the tests and isolating the environment are now first-class.
- **Subagent orchestration.** One mechanic dominates: a lead delegates self-contained work to subagents that run in isolated context windows and return only a distilled summary. The orchestrator-worker pattern wins decisively on breadth-first *research* (Anthropic measured +90.2% over a single agent) but at **~15× the token cost**, and it does **not** transfer to implementation — parallel writers make conflicting implicit decisions that can't be reconciled at merge. Default to a single linear agent for coding; reserve parallelism for read-heavy research/review, isolated by disjoint file ownership or git worktrees.
- **Workstream logging.** Because every session starts with a fresh context window, auditability and resumability come from layered artifacts, not any single feature: an append-only progress journal read at start and updated at end, a machine-checkable task ledger with explicit done/next markers, an indexed memory store, git/PR discipline, named resumable sessions, and lifecycle hooks + OpenTelemetry for the audit trail. Crucially, **nothing auto-persists a work log** — the discipline of writing state to files is what makes autonomous runs resumable, and completion must be gated on a recorded check, never self-asserted.
- **Permissions & security.** Safety is defense-in-depth: deny-first permission rules, a task-matched permission/approval mode, OS-level sandboxing that enforces **both filesystem and network** boundaries, and an egress allowlist plus credential scrubbing. The mental model that matters:

permission rules gate the model's *decision*, while the sandbox enforces the *boundary* at the OS level even when the model is compromised — neither alone is sufficient. Prompt injection remains unsolved (~1% residual attack success), so treat all tool and web output as untrusted data, lean on the sandbox + network allowlist as the true enforcement layer, and reserve YOLO/bypass modes for disposable, isolated environments with trusted code only.

The biggest shifts for 2026

1. **Prompt engineering → harness/loop engineering.** The differentiator is now the control loop, verification, and containment you build — not the wording of a prompt.
2. **Runaway cost is the #1 operational risk.** Widely reported blowups (a single hobby task reaching ~$6.5k; a full multi-agent harness costing ~20× a solo agent) have made hard ceilings, kill switches, and token-velocity alerts mandatory — and exposed that Codex lacks an in-tool per-run cap.
3. **Reward hacking is swamping intelligence gains.** Cursor measured a frontier model falling from **87.1% → 73.0%** on SWE-bench Pro once upstream lookup and git-history mining were sealed. "Just add tests" is no longer enough; the tests themselves must be guarded.
4. **Verification left the author and left green tests.** Separate evaluator agents grade explicit rubrics and observe real end-to-end/browser behavior, because models confidently praise their own mediocre work.
5. **Multi-agent got quantified — and narrowed.** The 4×/15× token multiplier and the "don't parallelize writers" rule (from both Anthropic and Cognition) puncture the "swarm of agents" hype for coding.
6. **Sandboxing became table stakes,** and classifier-gated "auto mode" emerged as a middle path between manual approval and blanket bypass — though at an honest ~17% false-negative rate it is a *layer*, not a guarantee.

What should change in the best-practices repo

- **Adopt the enforcement stack as templates, not prose:** a committed deny-first `.claude/settings.json` (secret-file denies, MCP gate), a `PreToolUse` guard blocking test-file edits and destructive commands, a `PostToolUse` format/lint hook, and a `Stop`-hook test gate.
- **Standardize instructions:** `AGENTS.md` as the canonical file with an `@AGENTS.md` import in `CLAUDE.md`; enforce the <200-line / commands-first / boundaries-explicit rule; move volatile detail into path-scoped `.claude/rules/*.md` and skills.
- **Ship the long-run scaffolding pattern:** `init.sh`, an append-only progress journal, a `feature_list.json` ledger with a verify-gated `passes` flag, and commit-per-unit git discipline.
- **Bake in verification & anti-reward-hacking:** name the exact check commands, require evidence (pasted output/screenshots) over assertions, add a standing "never weaken or edit tests" rule, and prescribe environment isolation for evals.
- **Make containment the default:** enable the OS sandbox (filesystem *and* network) with an egress allowlist, add credential scrubbing/masking, put ceilings on unattended runs (SDK caps for Claude;

CI timeout + spend circuit breaker for Codex), and document the kill switches (`CLAUDE_CODE_DISABLE_CRON`, Codex `[features] hooks = false`).

- **Correct the stale facts the verification pass caught** (and add a "verify against your installed version" note): Codex `--full-auto` is deprecated (use `--sandbox workspace-write`); `codex --continue` isn't real (use `codex resume --last`); Codex profiles are now per-file (`~/.codex/<name>.config.toml`); the Claude memory tool is GA (no beta header); Claude agent-hook default timeout is 60s; Codex's Linux sandbox now defaults to bubblewrap with Landlock as legacy fallback; Codex docs moved to `learn.chatgpt.com`.

How this impacts day-to-day agent work

- You'll invest more up front in the **loop and the check** than in the prompt, and expect the agent to *run the check and show output* rather than claim "done."
- **Trust green status less.** Both a Codex run's exit status and a Claude Routine's status only mean "ran without an infrastructure error," not "the task succeeded" — always open the run.
- **Delegate research to subagents; keep implementation single-threaded.** Parallelize only read-heavy work, and only with disjoint file ownership or worktrees.
- **Curate context continuously:** `/clear` aggressively, keep sessions single-purpose, watch `/context`, and keep a progress file so any session resumes cleanly.
- **Treat every tool/web/file output as data, never instructions** — quote and escalate injection-like text instead of acting on it.
- **Budget and containment become routine:** unattended runs get a ceiling and a kill switch, sensitive work runs in a sandbox or throwaway container, and you re-verify tooling facts against your installed CLI because the surface changes weekly.

What changed in the repository

This run stood up the operating contracts and the shared doctrine that now govern every agent, and wired the source of truth into global configuration.

Area	Change
Claude contract	`Claude/OPERATING_CONTRACT.md` written — 11 sections, fully source-cited
Codex contract	`Codex/OPERATING_CONTRACT.md` written — 11 sections, fully source-cited
Shared doctrine	`principles.md`, `definition-of-done.md`, `workstream-logging.md` enriched with verified cross-platform practice
Knowledge base	`Research/knowledge-base/2026-07-12-findings.md` created — cited findings per topic
Engine state	8 topics recorded at high confidence; 45 open questions logged as backlog for future runs
Global propagation	`~/.claude/CLAUDE.md` and `~/.codex/AGENTS.md` now point every session at this repo

Topic coverage

Topic	Confidence	Verified practices
agentic-loops	high	11
hooks	high	10
session-management	high	11
repo-setup	high	11
verification-self-checking	high	11
subagent-orchestration	high	10
workstream-logging	high	10
permissions-security	high	11

All eight topics reached **high** confidence after adversarial verification: every research claim was re-checked against primary sources, and version-sensitive or unverifiable claims were demoted to the open-questions backlog rather than shipped into the contracts.

Impact on day-to-day agent work

An agent starting work in any repository now:

- **Runs a session-start ritual** — loads the doctrine and platform contract, orients via `git status` / `log`, reloads the workstream journal, and smoke-tests before writing new code.
- **Runs loops only with a verifiable end state and a hard ceiling** — `/goal` for condition-terminated work, `/loop` or the Monitor tool for interval/watch work, Routines for durable jobs; never terminating on "code generated," always bounded by turn/budget caps and a human checkpoint before anything irreversible.
- **Treats hooks, not prose, as the enforcement layer** — deterministic format/lint/protect/completion gates that fire regardless of model judgment.
- **Gates "done" on observed, recorded evidence** — exercised at the real surface a user reaches, adversarially self-reviewed, edge cases covered; never self-asserted.
- **Logs every unit of work** into an indexed, resumable workstream journal.
- **Operates under least privilege** — sandboxed and egress-aware, treating all tool and web output as untrusted data, never as instructions.

Open questions carried forward

Adversarial verification surfaced **45 open questions** — the version-sensitive and still-unresolved items the next weekly runs should chase. One representative item per topic:

- **agentic-loops** — Codex has no documented per-run turn/budget-cap flag analogous to Claude's maxTurns/maxBudgetUsd - it relies on sandbox/approval limits, account usage limits, reasoning_effort, and human review. Is there a supported way to hard-cap a single codex exec run's token/dollar spend from the CLI, or must that be enforced externally (CI timeout / spend circuit breaker)?
- **hooks** — Exact delivery mechanism and full field schema of Codex CLI's `notify` program payload (argv positional argument vs stdin; whether the `type` value is 'agent-turn-complete' and what other fields like turn-id / input-messages / last-assistant-message are included) — the official advanced-config page was unreachable at fetch time, so this should be verified against the installed Codex version.
- **session-management** — The exact context-utilization threshold that triggers automatic compaction in Claude Code is not officially published and varies by model and window size (e.g., 200K vs 1M / Sonnet 5); practitioner figures (~70-80%) are unverified against Anthropic docs.
- **repo-setup** — Claude Code still does not natively read AGENTS.md as of mid-2026 (requires an @AGENTS.md import or symlink); the team has signaled openness to standardizing once features stabilize, so native support could land and change the recommended unification pattern — re-verify against code.claude.com/docs/en/memory.
- **verification-self-checking** — Whether the Codex apply_patch file-writer reliably triggers PreToolUse/PostToolUse hooks on all current versions. Official docs now say hooks fire for apply_patch, but an earlier gap (GitHub issue openai/codex #16732) let the agent edit test files via the patch tool despite Bash-only hook guards — so test-edit-blocking hooks may not be airtight depending on version.
- **subagent-orchestration** — Codex enablement specifics: the official docs describe enabling multi-agents via the `/experimental` toggle and an `[agents]` config section, but the exact `[features] multi_agent = true` key and the default `job_max_runtime_seconds` value (practitioner posts say 1800s; one official extract listed the key without a number) should be confirmed against the current official Codex config.toml reference before relying on them.
- **workstream-logging** — No cross-tool standard for the work-journal itself has emerged as of mid-2026 — teams roll their own (claude-progress.txt vs NOTES.md vs feature_list.json vs tasks/todo.md), so portability between Claude Code and Codex depends on self-imposed conventions rather than a shared spec.
- **permissions-security** — Prompt injection is not solved: Anthropic cites ~1% residual attack success and a measured ~17% false-negative rate for auto mode on real overeager actions, so no configuration makes an autonomous agent safe on untrusted input without OS-level containment.

The full list is tracked in `Research/engine/state.json` and drives the scope of the next run.

Method & provenance

- **24 agents, zero errors:** 8 topics × (research → adversarial verify) + 3 platform deep-dives + 5 synthesis agents.
- **~2.86M tokens**, 423 tool calls, ~29 minutes wall-clock.

- Every non-obvious mechanic in the contracts is cited inline to a primary source (Anthropic / OpenAI docs, changelogs, engineering blogs).
- Fully re-runnable and self-resuming via `Research/engine/research-workflow.js` and the `/weekly-research` command.